

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

BÁO CÁO BÀI TẬP LỚN
Matrix Chain Multiplication

Quy hoạch động trên đoạn - ba lời giải đối chiếu

Giảng viên hướng dẫn: TS. Huỳnh Thị Thanh Thương

Lớp: CS112.Q25

Họ và Tên MSSV

Hoàng Quốc Duy 24520373

Thành phố Hồ Chí Minh, tháng 6 năm 2026

Mục lục

1	Giới thiệu bài toán	1
1.1	Vì sao thứ tự nhân lại quan trọng	1
1.2	Phát biểu hình thức	1
1.3	Quy mô không gian tìm kiếm	1
1.4	Phạm vi	1
2	Thuật toán sử dụng	2
2.1	Cấu trúc con tối ưu	2
2.2	Công thức truy hồi	2
2.3	Ba lời giải	2
2.4	Mã giả	3
2.5	Cài đặt Python	4
2.6	Kiểm tra chéo nội bộ	5
3	Ví dụ minh họa thuật toán	5
3.1	Lấp bảng theo độ dài đoạn	5
3.2	Hai bảng đầy đủ	6
3.3	Truy vết dựng dấu ngoặc	6
4	Phân tích độ phức tạp	7
4.1	Thời gian và bộ nhớ của bản bottom-up	7
4.2	Vì sao đệ quy thuần thất bại	7
5	Cài đặt và thiết kế thử nghiệm	8
5.1	Tổ chức mã nguồn	8
5.2	Hai lớp kiểm thử	8
5.3	Cách xây dựng giá trị kỳ vọng	9
5.4	Cách đánh giá đúng/sai	9
5.5	Quy mô thử nghiệm	9
6	Kết quả thử nghiệm và đánh giá	10
6.1	Kết quả kiểm thử	10
6.2	Đầu ra mẫu của CLI	10
6.3	Đánh giá hiệu năng	10
7	Mô tả chức năng và cách thao tác	11
7.1	Giao diện dòng lệnh	11
7.2	Giao diện web demo	11
8	Tính năng mở rộng	13
8.1	Ba lời giải đối chiếu	13
8.2	Giao diện nhập liệu trực quan	13

8.3	Triển khai trực tuyến	14
9	So sánh với công cụ AI	14
10	Kết luận	15
10.1	Kết quả đạt được	15
10.2	Hạn chế	15
10.3	Hướng mở rộng	15
	Tài liệu tham khảo	15

1 Giới thiệu bài toán

1.1 Vì sao thứ tự nhân lại quan trọng

Nhân hai ma trận A cỡ $p \times q$ với B cỡ $q \times r$ cho ra ma trận $p \times r$ và tốn đúng $p \cdot q \cdot r$ phép nhân vô hướng. Phép nhân ma trận có tính kết hợp: $(AB)C = A(BC)$. Kết quả cuối cùng không đổi, nhưng số phép nhân để đi tới kết quả thì phụ thuộc vào cách ta gom nhóm.

Lấy ba ma trận A_1 (10×100), A_2 (100×5), A_3 (5×50):

- $((A_1 A_2) A_3)$: $10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 5000 + 2500 = \mathbf{7500}$.
- $(A_1 (A_2 A_3))$: $100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 25000 + 50000 = \mathbf{75000}$.

Hai cách cùng cho một tích nhưng lệch nhau gấp mười lần. Khi chuỗi dài, khoảng cách này còn lớn hơn nữa. Đó là lý do cần một thuật toán chọn thứ tự nhân tối ưu thay vì nhân tuần tự từ trái sang phải.

1.2 Phát biểu hình thức

Cho chuỗi n ma trận $A_1, A_2, \dots, A_n$, trong đó A_i có kích thước $p_i \times p_{i+1}$. Toàn bộ kích thước được mô tả bằng mảng $p = [p_0, p_1, \dots, p_n]$ gồm $n + 1$ phần tử (ở đây dùng chỉ số gốc 0, nên A_i là $p_i \times p_{i+1}$ với $0 \leq i \leq n - 1$).

Định nghĩa 1 (Bài toán MCM). *Tìm cách đặt dấu ngoặc đầy đủ cho tích $A_1 A_2 \cdots A_n$ sao cho tổng số phép nhân vô hướng là nhỏ nhất.*

1.3 Quy mô không gian tìm kiếm

Gọi $P(n)$ là số cách đặt dấu ngoặc cho n ma trận. Cách đặt ngoặc ngoài cùng chia chuỗi thành hai phần tại một vị trí k nào đó, nên

$$P(1) = 1, \quad P(n) = \sum_{k=1}^{n-1} P(k) P(n-k).$$

Đây chính là truy hồi của số Catalan: $P(n) = C_{n-1}$, và $C_{n-1} = \Omega(4^n/n^{3/2})$ tăng theo hàm mũ. Duyệt mọi cách đặt ngoặc là bất khả thi ngay với n vừa phải. Mục 4 đo cụ thể sự bùng nổ này.

1.4 Phạm vi

Bài tập tập trung vào *thuật toán thuần*: chỉ tính chi phí tối thiểu và thứ tự đặt ngoặc, không thực hiện phép nhân ma trận thật và không dùng thư viện ma trận. Hệ thống cần: nhận và kiểm tra mảng kích thước; tính bảng chi phí và bảng vị trí cắt; truy vết để dựng dấu ngoặc

tối ưu; in bảng cùng đáp số. Ngoài phần lõi còn có giao diện dòng lệnh và một giao diện web minh hoạ từng bước.

2 Thuật toán sử dụng

2.1 Cấu trúc con tối ưu

Quan sát mẫu chốt: trong cách đặt ngoặc tối ưu của $A_i \cdots A_j$, phép nhân cuối cùng tách chuỗi tại một vị trí k thành $(A_i \cdots A_k)$ và $(A_{k+1} \cdots A_j)$. Nếu một trong hai phần đó *không* được đặt ngoặc tối ưu, ta có thể thay bằng cách tốt hơn để giảm tổng chi phí – mâu thuẫn. Vậy lời giải tối ưu của bài toán lớn chứa lời giải tối ưu của các bài toán con: đây là điều kiện *cấu trúc con tối ưu*, tiền đề để áp dụng quy hoạch động [1, 2].

2.2 Công thức truy hồi

Gọi $\text{cost}[i][j]$ là số phép nhân vô hướng tối thiểu để nhân đoạn $A_i \cdots A_j$ (chỉ số 1-based theo CLRS, $1 \leq i \leq j \leq n$). Ma trận A_i có kích thước $p[i-1] \times p[i]$ với p là mảng kích thước 0-based gồm $n+1$ phần tử. Do đó

$$\text{cost}[i][j] = \begin{cases} 0 & \text{nếu } i = j, \\ \min_{i \leq k < j} \left\{ \text{cost}[i][k] + \text{cost}[k+1][j] + p[i-1] \cdot p[k] \cdot p[j] \right\} & \text{nếu } i < j. \end{cases}$$

Đáp số là $\text{cost}[1][n]$. Bảng $\text{split}[i][j]$ lưu vị trí k đạt cực tiểu, dùng để truy vết lại dấu ngoặc.

Quy ước chỉ số. Code tuân theo đúng ký hiệu của Cormen: ma trận $A_1 \dots A_n$ (1-based), bảng kích thước $(n+1) \times (n+1)$ với padding tại index 0, công thức $p[i-1] \cdot p[k] \cdot p[j]$ [1]. DPV [2] dùng ký hiệu tương đương.

2.3 Ba lời giải

Để thấy rõ vai trò của quy hoạch động, hệ thống cài đặt ba lời giải cho cùng công thức trên.

(1) Đệ quy thuần. Dịch trực tiếp công thức truy hồi thành đệ quy, không lưu lại kết quả đoạn con. Cùng một đoạn bị tính lại rất nhiều lần nên số lần gọi tăng theo hàm mũ. Bản này chỉ dùng để minh hoạ, đúng như cách GeeksforGeeks và CP-Algorithms giới thiệu trước khi chuyển sang quy hoạch động.

(2) Đệ quy có ghi nhớ (top-down). Vấn đề quy như trên nhưng lưu kết quả mỗi đoạn (i, j) vào bộ nhớ đệm. Mỗi đoạn chỉ tính một lần, có $O(n^2)$ đoạn, mỗi đoạn quét $O(n)$ vị trí cắt, cho độ phức tạp $O(n^3)$.

(3) Lắp bảng từ dưới lên (bottom-up). Tính các đoạn theo *độ dài tăng dần*. Đây là bản chính, có truy vết dấu ngoặc và được dùng cho CLI lẫn web.

2.4 Mã giả

Điểm khó hình dung nhất của bài toán là thứ tự lắp bảng: không lắp theo hàng hay cột mà theo *độ dài đoạn* $\ell = 2, 3, \dots, n$. Khi tính một đoạn dài ℓ , mọi đoạn ngắn hơn mà nó cần đều đã có sẵn [1, 2].

Thuật toán 1 Bottom-Up-MCM(p)

Vào: Mảng kích thước $p[0..n]$ ($n + 1$ phần tử, 0-based)

Ra: Bảng $\text{cost}[1..n][1..n]$ và $\text{split}[1..n][1..n]$

```

1:  $n \leftarrow \text{length}(p) - 1$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $\text{cost}[i][i] \leftarrow 0$ 
4: end for
5: for  $\ell \leftarrow 2$  to  $n$  do                                     ▷  $\ell$ : độ dài đoạn
6:   for  $i \leftarrow 1$  to  $n - \ell + 1$  do
7:      $j \leftarrow i + \ell - 1$ 
8:      $\text{cost}[i][j] \leftarrow \infty$ 
9:     for  $k \leftarrow i$  to  $j - 1$  do
10:       $q \leftarrow \text{cost}[i][k] + \text{cost}[k + 1][j] + p[i - 1] \cdot p[k] \cdot p[j]$ 
11:      if  $q < \text{cost}[i][j]$  then
12:         $\text{cost}[i][j] \leftarrow q$ 
13:         $\text{split}[i][j] \leftarrow k$ 
14:      end if
15:    end for
16:  end for
17: end for
18: return  $\text{cost}, \text{split}$ 

```

Thuật toán 2 Build-Parens(split, i, j)

```

1: if  $i = j$  then
2:   return  $?A_i?$ 
3: else
4:    $k \leftarrow \text{split}[i][j]$ 
5:   return  $?(? \parallel \text{Build-Parens}(\text{split}, i, k) \parallel \text{Build-Parens}(\text{split}, k + 1, j) \parallel ?)?$ 
6: end if

```

2.5 Cài đặt Python

Mã nguồn chia thành các module nhỏ, mỗi module một việc: `validation.py` (kiểm tra dữ liệu), `core.py` (ba lời giải + truy vết + kiểm tra chéo), `display.py` (định dạng bảng), `cli.py` (vào/ra dòng lệnh). Hai đoạn dưới trích nguyên văn từ `src/mcm2/core.py`.

```

1 def bottom_up(dims):
2     n = validate_dimensions(dims)
3     p = list(dims)
4
5     # Bảng kích thước (n+1) x (n+1); o index 0 là padding không dùng.
6     cost = [[0] * (n + 1) for _ in range(n + 1)]
7     split = [[0] * (n + 1) for _ in range(n + 1)]
8
9     # Đương chéo cost[i][i] = 0 cho mọi i = 1..n (đã sẵn bằng 0).
10    for i in range(1, n + 1):
11        cost[i][i] = 0
12
13    # Lấp bảng theo độ dài đoạn tăng dần (length = 2..n).
14    for length in range(2, n + 1):
15        for i in range(1, n - length + 2):
16            j = i + length - 1
17            best = INF
18            best_k = i
19            for k in range(i, j):
20                # Công thức CLRS 15.2: q =
21                # cost[i][k] + cost[k + 1][j] + p[i - 1] * p[k] * p[j]
22                q = cost[i][k] + cost[k + 1][j] + p[i - 1] * p[k] * p[j]
23                if q < best:
24                    best = q
25                    best_k = k
26            cost[i][j] = best
27            split[i][j] = best_k
28
29    return cost, split

```

Listing 1: Lấp bảng từ dưới lên – trích từ `core.py`

```

1 def optimal_parenthesization(split, i, j):
2     if i == j:

```

```

3         return f"A{i}"                                # ten ma tran 1-based theo CLRS
4     k = split[i][j]
5     left = optimal_parenthesization(split, i, k)
6     right = optimal_parenthesization(split, k + 1, j)
7     return f"({left}{right})"

```

Listing 2: Truy vết dấu ngoặc – trích từ `core.py`

2.6 Kiểm tra chéo nội bộ

Sau khi lấp bảng và dựng dấu ngoặc, hàm `solve` tính lại chi phí một lần nữa, lần này *chỉ dựa trên bảng split* (độc lập với bảng `cost`), rồi so với `cost[1][n]`. Nếu hai con số lệch nhau, chương trình báo lỗi ngay thay vì in kết quả sai.

3 Ví dụ minh họa thuật toán

Xét ví dụ kinh điển với $p = [30, 35, 15, 5, 10, 20, 25]$, tức $n = 6$ ma trận: $A_1 (30 \times 35)$, $A_2 (35 \times 15)$, $A_3 (15 \times 5)$, $A_4 (5 \times 10)$, $A_5 (10 \times 20)$, $A_6 (20 \times 25)$. Ví dụ này có trong cả CLRS lẫn slide bài giảng nên tiện đối chiếu.

3.1 Lấp bảng theo độ dài đoạn

Khởi tạo: $\text{cost}[i][i] = 0$ cho mọi $i = 1..6$.

$\ell = 2$ (đoạn 2 ma trận, chỉ một vị trí cắt $k = i$):

Đoạn	k	$q = p[i-1] \cdot p[k] \cdot p[j]$	cost	split
$A_1 A_2$	1	$p[0] \cdot p[1] \cdot p[2] = 30 \cdot 35 \cdot 15 = 15750$	15750	1
$A_2 A_3$	2	$p[1] \cdot p[2] \cdot p[3] = 35 \cdot 15 \cdot 5 = 2625$	2625	2
$A_3 A_4$	3	$p[2] \cdot p[3] \cdot p[4] = 15 \cdot 5 \cdot 10 = 750$	750	3
$A_4 A_5$	4	$p[3] \cdot p[4] \cdot p[5] = 5 \cdot 10 \cdot 20 = 1000$	1000	4
$A_5 A_6$	5	$p[4] \cdot p[5] \cdot p[6] = 10 \cdot 20 \cdot 25 = 5000$	5000	5

$\ell = 3$ (hai lựa chọn k , chọn cực tiểu – in đậm):

Đoạn	k	$q = \text{cost}[i][k] + \text{cost}[k+1][j] + p[i-1] \cdot p[k] \cdot p[j]$	cost	split
$A_1:A_3$	1	$0 + 2625 + 30 \cdot 35 \cdot 5 = \mathbf{7875}$	7875	1
	2	$15750 + 0 + 30 \cdot 15 \cdot 5 = 18000$		
$A_2:A_4$	2	$0 + 750 + 35 \cdot 15 \cdot 10 = 6000$	4375	3
	3	$2625 + 0 + 35 \cdot 5 \cdot 10 = \mathbf{4375}$		
$A_3:A_5$	3	$0 + 1000 + 15 \cdot 5 \cdot 20 = \mathbf{2500}$	2500	3
	4	$750 + 0 + 15 \cdot 10 \cdot 20 = 3750$		
$A_4:A_6$	4	$0 + 5000 + 5 \cdot 10 \cdot 25 = 6250$	3500	5
	5	$1000 + 0 + 5 \cdot 20 \cdot 25 = \mathbf{3500}$		

$\ell = 4, 5, 6$ (giá trị cực tiểu in đậm):

Đoạn	Các ứng viên q theo k	cost	split
$A_1:A_4$	$k=1:14875, k=2:21000, k=3:\mathbf{9375}$	9375	3
$A_2:A_5$	$k=2:13000, k=3:\mathbf{7125}, k=4:11375$	7125	3
$A_3:A_6$	$k=3:\mathbf{5375}, k=4:9500, k=5:10000$	5375	3
$A_1:A_5$	$k=1:28125, k=2:27250, k=3:\mathbf{11875}, k=4:15375$	11875	3
$A_2:A_6$	$k=2:18500, k=3:\mathbf{10500}, k=4:18125, k=5:24625$	10500	3
$A_1:A_6$	$k=1:36750, k=2:32375, k=3:\mathbf{15125}, k=4:21875, k=5:26875$	15125	3

3.2 Hai bảng đầy đủ

Sau khi lấp xong, hai bảng (nhân hàng/cột theo chỉ số 1-based như chương trình in ra) là:

Bảng chi phí							Bảng vị trí cắt					
	1	2	3	4	5	6		2	3	4	5	6
1	0	15750	7875	9375	11875	15125	1	1	1	3	3	3
2	-	0	2625	4375	7125	10500	2	-	2	3	3	3
3	-	-	0	750	2500	5375	3	-	-	3	3	3
4	-	-	-	0	1000	3500	4	-	-	-	4	5
5	-	-	-	-	0	5000	5	-	-	-	-	5
6	-	-	-	-	-	0						

3.3 Truy vết dựng dấu ngoặc

```

build_parens(split, 1, 6)          # đoạn A1..A6, 1-based
split[1][6] = 3 -> ( build(1,3) build(4,6) )
    split[1][3] = 1 -> ( A1 ( A2 A3 ) ) = (A1(A2A3))
    split[4][6] = 5 -> ( ( A4 A5 ) A6 ) = ((A4A5)A6)
-> ((A1(A2A3))((A4A5)A6))

```

Đáp số: chi phí tối thiểu $\text{cost}[1][6] = 15125$, dấu ngoặc tối ưu $((A1(A2A3))((A4A5)A6))$. Cả ba lời giải đều cho cùng con số 15125, và bộ kiểm thử đối chiếu thêm với liệt kê toàn bộ (mục 5).

4 Phân tích độ phức tạp

4.1 Thời gian và bộ nhớ của bản bottom-up

Ba vòng lặp lồng nhau theo ℓ, i, k cho tổng số phép tính

$$\sum_{\ell=2}^n (n - \ell + 1)(\ell - 1) = \Theta(n^3).$$

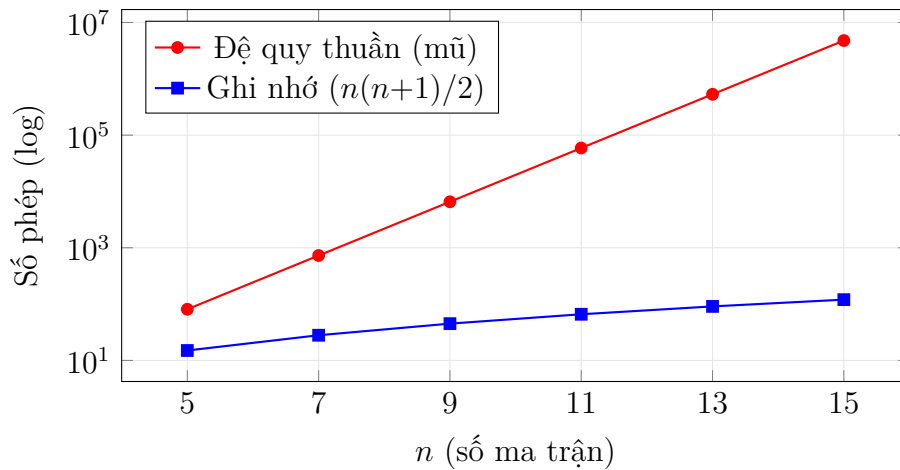
Hai bảng cost và split mỗi bảng $\Theta(n^2)$ ô, nên bộ nhớ phụ là $\Theta(n^2)$. Bản top-down có cùng $\Theta(n^3)$ thời gian và $\Theta(n^2)$ bộ nhớ vì cũng giải đúng $O(n^2)$ đoạn, mỗi đoạn quét $O(n)$ vị trí cắt.

4.2 Vì sao đệ quy thuần thất bại

Bản đệ quy thuần gọi lại cùng một đoạn rất nhiều lần. Đếm số lần gọi hàm cho thấy nó tăng theo cấp 3^{n-1} , trong khi bản ghi nhớ chỉ tính mỗi đoạn đúng một lần (tổng $n(n+1)/2$ đoạn). Bảng dưới là số liệu đo trực tiếp bằng `bench_naive.py`:

Bảng 1: Số lần gọi đệ quy: bản thuần so với bản ghi nhớ.

n	Gọi (đệ quy thuần)	Tính (ghi nhớ)	Số Catalan C_{n-1}
5	81	15	14
8	2 187	36	429
10	19 683	55	4 862
12	177 147	78	58 786
15	4 782 969	120	2 674 440



Hình 1: Trục tung thang log: bản thuần tăng theo hàm mũ, bản ghi nhớ tăng theo đa thức.

Chính sự bùng nổ này là động lực của quy hoạch động: lưu lại kết quả đoạn con để dùng lại, đổi bộ nhớ $\Theta(n^2)$ lấy thời gian từ cấp mũ xuống $\Theta(n^3)$.

5 Cài đặt và thiết kế thử nghiệm

5.1 Tổ chức mã nguồn

Phần lõi chỉ dùng thư viện chuẩn Python, chạy trên Python 3.10 trở lên. Toàn bộ hàm tính toán là hàm thuần (không in, không đọc ghi tệp); chỉ `cli.py` đụng tới vào/ra. Nhờ tách bạch như vậy, mỗi hàm có thể gọi trực tiếp trong test và so sánh giá trị trả về, không cần bắt chuỗi in ra màn hình.

5.2 Hai lớp kiểm thử

Hệ thống có hai lớp kiểm thử bổ trợ nhau.

(a) **Bộ testcase tường minh** nằm trong `testcases/`, gồm tệp dữ liệu `cases.json` và trình chấm `run_testcases.py`. Dữ liệu chia hai nhóm:

- *Nhóm hợp lệ* (12 ca): mỗi ca ghi mảng kích thước, chi phí kỳ vọng và dấu ngoặc kỳ vọng. Bốn ca đầu lấy từ ví dụ trong tài liệu và CLRS; tám ca còn lại thiết kế để phủ nhiều dạng cấu trúc: chuỗi hai ma trận (chỉ một cách nhân), trường hợp thứ tự ngoặc đổi chi phí gấp mười lần, kích thước tăng dần, giảm dần, bằng nhau, dao động mạnh, và hỗn hợp không quy luật.
- *Nhóm sai* (5 ca): phủ ba nhánh kiểm tra dữ liệu - mảng quá ngắn, phần tử không dương, phần tử không phải số nguyên - và đối chiếu thông điệp lỗi trùng khớp từng ký tự.

(b) **Bộ kiểm thử tự động** nằm trong `tests/`, chạy bằng `pytest`. Gồm test ví dụ cụ thể, test thông điệp lỗi, và một số test *dựa trên tính chất* (property-based). Vì yêu cầu chỉ dùng thư viện chuẩn nên thay cho thư viện sinh dữ liệu chuyên dụng, các test tính chất dùng `random.Random(seed)` với seed cố định, mỗi test sinh từ 150 đến 200 mẫu ngẫu nhiên và có thể tái lập nhờ seed.

5.3 Cách xây dựng giá trị kỳ vọng

Đây là điểm cần cẩn thận: nếu ?đáp án mẫu? tự đặt cũng sai thì test vô nghĩa. Vì vậy mọi chi phí kỳ vọng của ca tự thiết kế đều được đối chiếu độc lập bằng lời giải đệ quy thuần (`naive_min_cost`) - thuật toán này liệt kê toàn bộ cách đặt ngoặc nên chắc chắn cho cực tiểu thật. Với các ca có nhiều lời giải cùng chi phí (ví dụ ca mọi kích thước bằng nhau), dấu ngoặc kỳ vọng được ghi đúng theo dấu ngoặc mà bản bottom-up chọn, để test không phụ thuộc vào việc ?đoán? một trong nhiều đáp án tối ưu.

5.4 Cách đánh giá đúng/sai

- Ca hợp lệ: đúng khi *cả* chi phí lẫn dấu ngoặc khớp giá trị kỳ vọng.
- Ca sai: đúng khi chương trình ném ngoại lệ `DimensionError` với thông điệp trùng khớp nguyên văn.
- Test tính chất ?tối ưu thực sự?: so chi phí của bottom-up và top-down với chi phí liệt kê toàn bộ; đây là kiểm chứng mạnh nhất vì xác nhận lời giải đạt cực tiểu toàn cục, không chỉ tự nhất quán.
- Test tính chất ?dấu ngoặc hợp lệ?: kiểm tra ngoặc cân bằng ở mọi tiền tố và các tên $A_1, \dots, A_n$ xuất hiện đúng thứ tự.

5.5 Quy mô thử nghiệm

- Bộ testcase tường minh: 17 ca (12 hợp lệ + 5 sai).
- Bộ `pytest`: 42 test. Trong đó các test tính chất sinh ngẫu nhiên cộng dồn khoảng 1000 mẫu. Riêng test đối chiếu liệt kê toàn bộ chạy với $n \leq 6$ (mỗi mẫu so tới $C_5 = 42$ cách đặt ngoặc).
- Ba lời giải được kiểm tra cho ra cùng kết quả trên mọi ví dụ bất buộc.

6 Kết quả thử nghiệm và đánh giá

6.1 Kết quả kiểm thử

Toàn bộ 42 test pytest đều PASS (đo trên Python 3.13, khoảng 0,2 giây). Bộ testcase tường minh đạt 17/17 PASS. Kết quả dòng lệnh trùng khớp với bảng tham chiếu trong tài liệu.

6.2 Đầu ra mẫu của CLI

Lệnh python -m mcm2 30 35 15 5 10 20 25 cho:

Bảng chi phí (số phép nhân tối thiểu của từng đoạn):

	1	2	3	4	5	6
1	0	15750	7875	9375	11875	15125
2	-	0	2625	4375	7125	10500
3	-	-	0	750	2500	5375
4	-	-	-	0	1000	3500
5	-	-	-	-	0	5000
6	-	-	-	-	-	0

Bảng vị trí cắt tối ưu:

	1	2	3	4	5	6
1	-	1	1	3	3	3
2	-	-	2	3	3	3
3	-	-	-	3	3	3
4	-	-	-	-	4	5
5	-	-	-	-	-	5
6	-	-	-	-	-	-

Dấu ngoặc tối ưu: ((A1(A2A3))((A4A5)A6))

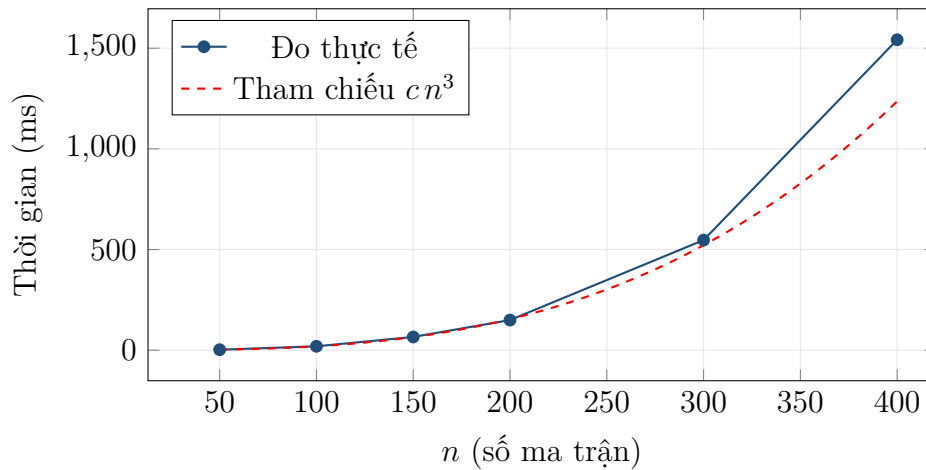
Số phép nhân vô hướng tối thiểu: 15125

6.3 Đánh giá hiệu năng

Bảng và biểu đồ dưới đo thời gian chạy của bottom-up theo n (lấy giá trị nhỏ nhất qua 5 lần chạy, dữ liệu sinh từ seed cố định, đo bằng `benchmark.py`):

Bảng 2: Thời gian thực thi bottom-up theo kích thước n .

n	Thời gian	Ghi chú
50	$\approx 2,3$ ms	
100	≈ 19 ms	gấp đôi n so với 50: tăng $\approx 8,3$ lần
150	≈ 66 ms	
200	≈ 150 ms	
300	≈ 547 ms	
400	≈ 1542 ms	gấp đôi n so với 200: tăng $\approx 8,3$ lần



Hình 2: Thời gian đo bám sát đường tham chiếu $\Theta(n^3)$.

Khi n tăng từ 50 lên 100, thời gian tăng khoảng 8,3 lần - gần với dự đoán lý thuyết $2^3 = 8$ lần của $\Theta(n^3)$. Với $n > 100$, chương trình vẫn chạy đúng và trả kết quả mà không đặt giới hạn cứng; phần truy vết đệ quy nâng giới hạn đệ quy khi n rất lớn để tránh tràn ngăn xếp.

7 Mô tả chức năng và cách thao tác

7.1 Giao diện dòng lệnh

Cách nhập: truyền mảng kích thước qua tham số dòng lệnh hoặc qua `stdin`.

```
python -m mcm2 30 35 15 5 10 20 25          # in ra hai bảng
python -m mcm2 30 35 15 5 10 20 25 --tables=cost # chỉ bảng chi phí
python -m mcm2 30 35 15 5 10 20 25 --tables=split # chỉ bảng vị trí cắt
python -m mcm2 30 35 15 5 10 20 25 --tables=None # chỉ kết quả
echo 30 35 15 5 10 20 25 | python -m mcm2      # nhập qua stdin
```

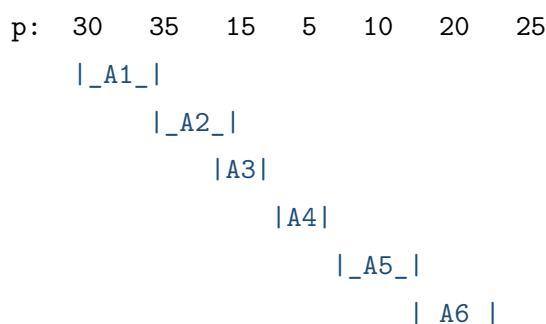
Các output trả về: tùy cờ `--tables`, chương trình in bảng chi phí, bảng vị trí cắt (ô không dùng để dấu -, các cột canh thẳng hàng), dấu ngoặc tối ưu, và số phép nhân vô hướng tối thiểu. Khi dữ liệu sai, chương trình in thông điệp lỗi tiếng Việt ra `stderr` và trả mã thoát khác 0 (1 cho lỗi dữ liệu, 2 cho lỗi nội bộ).

7.2 Giao diện web demo

Ứng dụng web (`web_app.py`, dùng Flask) có một endpoint `GET /` trả trang chính, và một endpoint `POST /api/solve` nhận JSON `{dims: [...]}`, giải bằng bottom-up rồi trả về: số ma trận, chi phí tối thiểu, dấu ngoặc, số cách đặt ngoặc (Catalan), bảng chi phí, bảng vị trí cắt, và danh sách các bước lắp bảng.

Cách nhập dữ liệu. Phần phức tạp nhất với người mới là hiểu mảng p : nó không phải kích thước của từng ma trận mà là *chuỗi các mốc kích thước nối tiếp*, trong đó số cột của một ma trận đồng thời là số hàng của ma trận sau. Để gỡ rào cản này, giao diện không bắt người dùng gõ thẳng mảng phẳng mà cung cấp một *bộ dựng theo từng ma trận*:

- Mỗi ma trận là một dòng A_i : $[hàng] \times [cột]$. Người dùng thêm/bớt ma trận bằng nút $+$ / $-$.
- Ô *số hàng* của mọi ma trận (trừ ma trận đầu) bị khoá và **tô đỏ**, tự động lấy bằng số cột của ma trận liền trước. Nhờ vậy ràng buộc $?cột trước = hàng sau?$ luôn được giữ và không thể nhập chuỗi không hợp lệ.
- Phần xem trước hiển thị song song *?Chuỗi ma trận?* (ví dụ $A_1(30 \times 35) \cdot A_2(35 \times 15) \dots$) và *?Mảng kích thước p ? tương ứng*, để người dùng thấy rõ mối liên hệ giữa hai cách biểu diễn.
- Một khối giải thích thu gọn (bấm để mở) trình bày quy tắc $A_i = p[i-1] \times p[i]$ kèm sơ đồ minh hoạ mỗi ma trận trải dưới hai mốc kích thước liền kề (Hình 3).
- Mục *?nhập nhanh?* cho người dùng thành thạo dán thẳng mảng p phẳng; hệ thống tự tách thành các ma trận tương ứng.



Hình 3: Sơ đồ trong khối giải thích: mỗi ma trận A_i nằm giữa hai mốc kích thước $p[i-1]$ và $p[i]$; con số 35 vừa là cột của A_1 vừa là hàng của A_2 nên chỉ ghi một lần.

Các tab kết quả. Giao diện (HTML và JavaScript thuần) chia ba tab:

- Bảng chi phí & cắt:** hai bảng đầy đủ; ô đáp số $cost[1][n]$ tô nổi bật, ô không dùng để dấu $-$.
- Các bước lắp bảng:** mỗi bước ứng với một đoạn (i, j) , liệt kê mọi vị trí cắt k kèm công thức đầy đủ và đánh dấu lựa chọn tối ưu. Có nút *Trước* / *Sau* để duyệt; ô đang tính được tô sáng trên bảng chi phí.
- Cây dấu ngoặc:** dựng biểu thức dấu ngoặc tối ưu từ bảng vị trí cắt.

Cách chạy web demo:

```
# 1. Tạo môi trường ảo (khuyến nghị)
python -m venv venv
```

```

venv\Scripts\activate

# 2. Cài Flask
pip install flask

# 3. Chạy server, mở trình duyệt vào địa chỉ in ra
python web_app.py          # mặc định http://127.0.0.1:5001

```

Với dữ liệu mẫu $p = [30, 35, 15, 5, 10, 20, 25]$, giao diện hiển thị chi phí 15125, dấu ngoặc $((A1(A2A3))((A4A5)A6))$, số cách đặt ngoặc 42, và 15 bước lấp bảng.

8 Tính năng mở rộng

Ngoài yêu cầu cốt lõi, dự án bổ sung một số tính năng nhằm nâng cao trải nghiệm học tập và kiểm chứng kết quả.

8.1 Ba lời giải đối chiếu

Thay vì chỉ cài một lời giải bottom-up, hệ thống cung cấp ba cách giải độc lập cùng tính toán trên một công thức truy hồi, để người dùng đối chiếu:

- **Đệ quy thuần** (`naive_min_cost`): dịch trực tiếp công thức sang đệ quy, không lưu kết quả đoạn con. Số lần gọi tăng theo hàm mũ ($\sim 3^{n-1}$). Dùng để minh họa vì sao cần quy hoạch động và để làm đối chứng tính tối ưu cho $n \leq 6$.
- **Đệ quy có ghi nhớ** (`memoized_min_cost`): dùng `lru_cache` để mỗi đoạn chỉ tính một lần; $O(n^3)$ thời gian, tính theo nhu cầu (top-down).
- **Lấp bảng từ dưới lên** (`bottom_up`): bản chính; $O(n^3)$ thời gian, $O(n^2)$ bộ nhớ, trả về đủ bảng để truy vết và minh họa.

Bộ kiểm thử xác nhận cả ba cho cùng chi phí trên mọi ví dụ, và bản đệ quy thuần làm đối chứng *tối ưu toàn cục*: `cost[1][n]` của bottom-up bằng chi phí nhỏ nhất khi liệt kê toàn bộ C_{n-1} cách đặt ngoặc.

8.2 Giao diện nhập liệu trực quan

Điểm khó với người mới là hiểu mảng p : nó không phải kích thước của từng ma trận mà là *chuỗi mốc kích thước nối tiếp* – con số ở giữa vừa là số cột của ma trận trước vừa là số hàng của ma trận sau. Giao diện web giải quyết bằng *bộ dựng theo từng ma trận*:

- Mỗi ma trận hiển thị dưới dạng A_i : `[hàng] × [cột]`; thêm/bớt bằng nút $+/−$.
- Ô số hàng của mọi ma trận (trừ ma trận đầu) bị khoá và tô đỏ, tự lấy bằng số cột của ma trận liền trước – đảm bảo ràng buộc “cột trước = hàng sau” và không thể nhập

chuỗi không hợp lệ.

- Xem trước hiển thị song song chuỗi ma trận (ví dụ $A_1(30 \times 35) \cdot A_2(35 \times 15) \dots$) và mảng p tương ứng.
- Khối giải thích thu gọn (bấm để mở) trình bày quy tắc $A_i = p[i-1] \times p[i]$ kèm sơ đồ minh họa mỗi ma trận trải dưới hai móc liền kề.

8.3 Triển khai trực tuyến

Ứng dụng web được triển khai công khai tại:

<https://matrix-chain-multiplication.onrender.com>

Máy chủ dùng **Render.com** (nền tảng PaaS), chạy Flask qua Gunicorn. Cấu hình khai báo trong `render.yaml`: build bằng `pip install -r requirements.txt && pip install -e .`, start bằng `gunicorn web_app:app`. Người dùng không cần cài đặt gì, mở trình duyệt và nhập thẳng mảng kích thước.

9 So sánh với công cụ AI

Để đánh giá độ tin cậy, nhóm thử cho một trợ lý AI (ChatGPT) giải cùng các ví dụ và so với hệ thống.

- **Với ví dụ kinh điển** $p = [30, 35, 15, 5, 10, 20, 25]$: AI cho đúng đáp số 15125 và dấu ngoặc tối ưu, kèm giải thích công thức truy hồi. Ví dụ này phổ biến trong tài liệu nên AI trả lời chính xác và đáng tin trong trường hợp này.
- **Với dữ liệu lạ** (ví dụ $p = [13, 5, 89, 3, 34]$, đáp số đúng là 2856): AI thường ra đúng chi phí nhưng không phải lúc nào cũng kèm bảng chi phí và bảng vị trí cắt đầy đủ, đôi khi bước trung gian trình bày thiếu nhất quán. Người dùng khó kiểm chứng nếu không tự tính lại, và AI không tự chạy lại để xác nhận.
- **Ưu thế của hệ thống**: (i) kết quả *tất định, tái lập được* - cùng dữ liệu luôn cho cùng kết quả; (ii) có *kiểm tra chéo nội bộ*, từ chối in ra kết quả không nhất quán; (iii) có *ba lời giải* đối chiếu lẫn nhau cùng bộ testcase và liệt kê toàn bộ làm đối chứng minh bạch; (iv) *hiển thị từng bước* candidate và cây dấu ngoặc, phục vụ mục tiêu học tập.

Tóm lại, AI tiện để giải nhanh ví dụ phổ biến, nhưng với yêu cầu chính xác, kiểm chứng được và minh họa từng bước thì một cài đặt thuật toán tường minh như hệ thống này đáng tin cậy hơn. Cách kiểm chứng an toàn là dùng kết quả của AI như một gợi ý rồi đối chiếu lại bằng chương trình.

10 Kết luận

10.1 Kết quả đạt được

Báo cáo trình bày trọn vẹn quá trình giải bài toán Matrix Chain Multiplication: từ trực giác ?thứ tự nhân quyết định chi phí?, cấu trúc con tối ưu, công thức truy hồi, mã giả, đến cài đặt Python. Hệ thống có ba lời giải (đệ quy thuần, đệ quy ghi nhớ, lấp bảng từ dưới lên) đối chiếu lẫn nhau, một bộ testcase tường minh 17 ca, một bộ pytest 42 test, một giao diện dòng lệnh và một giao diện web minh hoạ từng bước.

10.2 Hạn chế

- Chỉ tính chi phí và thứ tự đặt ngoặc, chưa thực hiện phép nhân ma trận thật.
- Web demo dùng máy chủ phát triển của Flask, chưa cấu hình máy chủ cho triển khai thực tế.
- Chưa cài thuật toán dưới bậc ba (ví dụ Hu-Shing $O(n \log n)$); ràng buộc này có chủ ý để bám sát phương pháp giảng dạy.

10.3 Hướng mở rộng

- Thêm hoạt cảnh đồng bộ giữa từng bước và ô tương ứng trên bảng.
- Cho phép xuất các bước ra tệp để chấm tự động.
- Bổ sung phép nhân ma trận thật để minh hoạ trọn vẹn từ đầu đến cuối.

Tài liệu

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, Third Edition. The MIT Press, 2009. ISBN 978-0-262-03384-8. (Chương 15: Dynamic Programming, mục 15.2: Matrix-chain multiplication, trang 370–378.)
- [2] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani. *Algorithms*. Bản thảo, 2006. <http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf> (Chương 6: Dynamic Programming, mục 6.5: Chain matrix multiplication.)
- [3] Huỳnh Thị Thanh Thương. *Bài giảng 16: Thiết kế thuật toán – Kỹ thuật Quy hoạch động*, môn Phân tích và Thiết kế Thuật toán (CS112), mã học phần C3T13. Trường Đại học Công nghệ Thông tin, ĐHQG TP.HCM, 30/05/2023. (Bốn ví dụ kiểm thử bắt buộc V01-V04 và mã giả tham chiếu.)